Matthew DiCicco AMS-530 Project 2

Matthew DiCicco

October 2026

1 Problem 2.1: MPI Collective Operations

1.1 MPI_Bcast

Purpose

The MPI_Bcast operation distributes data from a designated root process to all other processors/ranks active on an allocation. My implementation uses a binomial-tree approach, where processors that have received the data forward it to other processors, doubling the number of informed processors/ranks each communication round.

Pseudocode

Algorithm 1 Tree-Based Broadcast

Require: Data array A of size N, root rank r, number of processes P

$q \leftarrow (\text{rank} - r + P) \bmod P$
$mask \leftarrow 1$
while $mask < P$ **do**
    if $q < mask$ **then**
        $dest \leftarrow q + mask$
        if $dest < P$ **then**
            Send A to process $(dest + r) \bmod P$
        end if
    else if $q < 2 \cdot mask$ **then**
        $src \leftarrow q - mask$
        Receive A from process $(src + r) \bmod P$
    end if
    $mask \leftarrow 2 \cdot mask$
end while

Scalability Consideration

A linear broadcast requires the root to send data individually to the other $P-1$ processors, resulting in $O(P)$ sequential communication steps.

In contrast, the tree-based approach allows multiple processors/ranks to send simultaneously, requiring only $O(\log_2 P)$ communication rounds. Therefore, the tree-based algorithm scales more efficiently as the number of processors/ranks increases, assuming communication can occur concurrently.

1.2 MPI_Scatter

Purpose

The MPI_Scatter operation distributes distinct portions of an array from a designated root process to all processes in a communicator. Each process receives a unique block of N elements from the root's original array of size PN. Our implementation uses a linear approach, where the root sequentially sends each process its corresponding block.

Pseudocode

Algorithm 2 Linear Scatter

Require: Root array A of size PN, block size N, root rank r, number of processes P
Ensure: Each process receives its corresponding block of N elements

```
if rank = r then
    for dest = 0 to P − 1 do
        chunk ← A[dest · N : (dest + 1) · N]
        if dest = r then
            Copy chunk into local receive buffer
        else
            Send chunk to process dest
        end if
    end for
else
    Receive N elements from process r
end if
```

Scalability Consideration

The linear scatter requires the root rank to perform $P-1$ sequential sends, each containing N elements. Its communication time can be approximated by

$$T_{\text{linear}} \approx (P-1)(\alpha + \beta N),$$

where α represents message startup latency and β represents the transmission cost per element.

A tree-based scatter reduces the number of communication rounds from $O(P)$ to $O(\log P)$ by allowing intermediate ranks to participate in distributing the data. However, unlike broadcast, each rank must receive a distinct block of

N elements. Therefore, the root rank must still transmit all $(P-1)N$ elements destined for the other ranks, regardless of the communication structure.

Thus, a tree-based scatter reduces the overhead of sequential message startups, but cannot eliminate the bandwidth required to distribute the data. For small messages, communication latency dominates, making the tree-based approach more scalable. For large messages, the root rank's outgoing bandwidth becomes the primary performance bottleneck.

1.3 MPI_Allgather

Purpose

The MPI_Allgather operation collects a block of N elements from every rank and distributes the complete collection to all P ranks. Each rank therefore finishes with the same array of PN elements, ordered by the rank that contributed each block.

Our implementation uses a ring-based approach. Each rank initially stores its own block, then repeatedly sends a block to its right neighbor while receiving another block from its left neighbor. The newly received block is forwarded in the following round. After $P-1$ rounds, every rank has received the contributions from all other ranks.

Pseudocode

Algorithm 3 Ring-Based Allgather

Require: Local array A of size N, rank r, number of ranks P
Ensure: Array B of size PN containing all rank contributions

Copy A into block r of B
$right \leftarrow (r+1) \bmod P$
$left \leftarrow (r-1+P) \bmod P$
for $step = 0$ to $P-2$ **do**
    $sendBlock \leftarrow (r - step + P) \bmod P$
    $recvBlock \leftarrow (r - step - 1 + P) \bmod P$
    Simultaneously:
        Send block $sendBlock$ of B to $right$
        Receive block $recvBlock$ of B from $left$
end for

Scalability Consideration

A naive linear Allgather can be implemented by having each rank send its data directly to every other rank. This requires $P-1$ sends per rank and can create substantial network contention as the number of ranks increases. Alternatively, gathering all data at one root rank and then broadcasting it creates a centralized communication bottleneck.

My ring-based implementation distributes communication evenly across all ranks. In each of the $P-1$ rounds, every rank sends and receives exactly N elements. Assuming these exchanges occur concurrently, its communication time is approximately

$$T_{\text{ring}} \approx (P-1)(\alpha + \beta N),$$

where α is message startup latency and β is the transmission cost per element.

Another approach is recursive doubling which is more scalable for small messages, where communication latency dominates. It completes Allgather in $O(\log P)$ rounds, compared with the ring's $O(P)$ rounds. Thus, as the number of ranks increases, recursive doubling incurs significantly less message startup overhead.

For large messages, the ring is often more efficient because each rank continuously exchanges fixed-size blocks with its neighbors, allowing efficient use of network bandwidth. Both algorithms transfer $(P-1)N$ elements per rank, so neither reduces the total communication volume. Recursive doubling's advantage in communication rounds therefore becomes less important as data size increases.

In summary, recursive doubling favors latency-bound communication (small N, large P), while the ring favors bandwidth-bound communication (large N). The ring also supports arbitrary P, whereas basic recursive doubling requires a power of two.

1.4 MPI_Alltoall

Purpose

The MPI_Alltoall operation allows every rank to send a distinct block of N elements to every other rank. Each rank begins with PN elements, divided into P blocks according to their destination ranks, and finishes with PN elements received from all ranks, ordered by source.

Our implementation uses a pairwise exchange approach. In each round, every rank sends one block to a destination rank while simultaneously receiving another block from a source rank. After $P-1$ rounds, every rank has received all the blocks intended for it.

Pseudocode

Algorithm 4 Pairwise Alltoall

Require: Local send array A of size PN, block size N, rank r, number of ranks P

Ensure: Receive array B of size PN containing one block from every rank

```
1: Copy block r of A into block r of B
2: for step = 1 to P − 1 do
3:     dest ← (r + step) mod P
4:     src ← (r − step + P) mod P
5:     Simultaneously:
6:         Send block dest of A to rank dest
7:         Receive block src of B from rank src
8: end for
```

Scalability Consideration

The pairwise Alltoall requires $P-1$ communication rounds, with every rank sending and receiving N elements per round. Assuming concurrent exchanges, its communication time can be approximated by

$$T_{\text{pairwise}} \approx (P-1)(\alpha + \beta N),$$

where α is message startup latency and β is the transmission cost per element.

Unlike a centralized approach, which gathers and redistributes data through a root rank, pairwise exchange distributes communication evenly across all ranks. This eliminates the root bottleneck and allows communication to occur concurrently.

However, Alltoall requires every rank to deliver a distinct block to every other rank. Consequently, each rank must transmit $(P-1)N$ elements, and the total communication volume across all ranks is $P(P-1)N = O(P^2N)$. This quadratic growth is the fundamental scalability limitation of Alltoall.

For small blocks, startup latency dominates, making the $O(P)$ communication rounds expensive as P increases. For large blocks, bandwidth and network contention dominate because many ranks exchange large amounts of data simultaneously. Thus, pairwise exchange removes the centralized bottleneck, but cannot eliminate the increasing communication demand inherent to Alltoall.

1.5 MPI_Reduce

Purpose

The MPI_Reduce operation combines data from all P ranks using a specified operation, such as addition, multiplication, minimum, or maximum, and stores the final result at a designated root rank.

Our implementation performs an element-wise sum using a tree-based reduction. Each rank begins with an array of N elements. In each communication round, pairs of ranks combine their partial sums, reducing the number of active ranks by approximately half. After $\lceil \log_2 P \rceil$ rounds, the root contains the sum of all P arrays.

Pseudocode

Algorithm 5 Tree-Based Sum Reduction

Require: Local array A of size N, root rank r, number of ranks P
Ensure: Root rank receives the element-wise sum of all arrays

$q \leftarrow (\text{rank} - r + P) \bmod P$
$S \leftarrow A$
$distance \leftarrow 1$
while $distance < P$ **do**
    if $q \bmod (2 \cdot distance) = 0$ **then**
        $sender \leftarrow q + distance$
        if $sender < P$ **then**
            Receive array T from rank $(sender + r) \bmod P$
            for $i = 0$ to $N - 1$ **do**
                $S[i] \leftarrow S[i] + T[i]$
            end for
        end if
    else
        $receiver \leftarrow q - distance$
        Send S to rank $(receiver + r) \bmod P$
        break
    end if
    $distance \leftarrow 2 \cdot distance$
end while
if rank $= r$ **then**
    Store S in the final receive array
end if

Scalability Consideration

A linear reduction requires the root rank to receive $P-1$ arrays sequentially, resulting in $O(P)$ communication steps. The root also performs $O(PN)$ arithmetic operations, creating a centralized computation and communication bottleneck.

Our tree-based reduction distributes both communication and addition across the ranks. Each round combines partial sums from pairs of ranks, requiring only $O(\log P)$ communication rounds. Assuming concurrent communication, its approximate execution time is

$$T_{\text{tree}} \approx \lceil \log_2 P \rceil (\alpha + \beta N + \gamma N),$$

where α is message startup latency, β is transmission cost per element, and γ is addition cost per element.

The defining advantage is that increasing the number of ranks does not proportionally increase the work performed by the root. Instead, the reduction work is distributed throughout the tree, allowing the algorithm to scale logarithmically with P rather than linearly.

For small arrays, communication latency dominates, so reducing the number of message rounds provides the greatest improvement. For large arrays, the cost of transmitting and adding N elements per round becomes more significant. Nevertheless, the tree remains more scalable than a linear reduction because it avoids concentrating all communication and arithmetic at the root.

"'latex

1.6 MY_Global_Min_Loc

Purpose and Implementation

The MY_Global_Min_Loc function identifies the rank holding the minimum value at each index across P distributed arrays of length N. The root rank returns an array of N ranks corresponding to these minima.

Our implementation uses a tree-based reduction. Each rank initially stores its local values and its own rank in two arrays. During each communication round, receiving ranks compare their accumulated minima with those received from another rank, retaining the smaller value and its originating rank at each index. Sending ranks then exit the reduction. The number of active ranks is approximately halved each round until the root holds the final minimum ranks.

Pseudocode

Algorithm 6 Tree-Based Global Minimum Location

Require: Local array A of length N, root rank r, number of ranks P
Ensure: Root returns array R containing the rank holding the minimum at each index

$q \leftarrow (\text{rank} - r + P) \bmod P$
$minValues \leftarrow A$
$minRanks[i] \leftarrow$ rank for all i
$distance \leftarrow 1$
while $distance < P$ **do**
    if $q \bmod (2 \cdot distance) = 0$ **then**
        $sender \leftarrow q + distance$
        if $sender < P$ **then**
            $source \leftarrow (sender + r) \bmod P$
            Receive $incomingValues$ from $source$
            Receive $incomingRanks$ from $source$
            for $i = 0$ to $N - 1$ **do**
                if $incomingValues[i] < minValues[i]$ **then**
                $minValues[i] \leftarrow incomingValues[i]$
                $minRanks[i] \leftarrow incomingRanks[i]$
                end if
            end for
        end if
    else
        $receiver \leftarrow q - distance$
        $dest \leftarrow (receiver + r) \bmod P$
        Send $minValues$ to $dest$
        Send $minRanks$ to $dest$
        break
    end if
    $distance \leftarrow 2 \cdot distance$
end while
if $\text{rank} = r$ **then**
    $R \leftarrow minRanks$
end if

Scalability Consideration

A linear implementation sends all $P - 1$ arrays directly to the root, which then performs approximately $(P - 1)N$ comparisons. This creates a bottleneck because the root must receive and compare every rank's data sequentially. Its communication and computation time scales as

$$T_{\text{linear}} \approx (P - 1)(\alpha + \beta N + \gamma N).$$

Our tree-based approach distributes comparisons across ranks. Each receiving rank combines two partial results before forwarding them, reducing the number of active ranks by approximately half each round. The root therefore participates in only $\lceil \log_2 P \rceil$ communication rounds rather than $P-1$ sequential exchanges.

Because each message contains both N minimum values and N originating ranks, the approximate execution time is

$$T_{\text{tree}} \approx \lceil \log_2 P \rceil (\alpha + 2\beta N + \gamma N),$$

where α represents message startup latency, β is transmission cost per integer, and γ is comparison cost per element.

The defining scalability advantage is that the tree reduces the critical path from $O(PN)$ to $O(N \log P)$ while preserving the same $O(PN)$ total comparison work. As P increases, the tree allows comparisons to occur concurrently rather than concentrating them at the root. This advantage becomes more significant for larger numbers of ranks and larger arrays.

Results and Performance

The implementation was tested on SeaWulf using MPI_Wtime to measure the maximum elapsed time across ranks. The timing includes the tree-based reduction but excludes the additional communication used to collect original arrays for verification and file output.

Table 1 summarizes four representative tests.

Table 1: MY_Global_Min_Loc Execution Times

P	N	Maximum Time (s)	Correctness
3	4	0.000102758	PASS
3	10	0.0000991821	PASS
4	4	0.000122309	PASS
4	10	0.000133514	PASS

All four tests produced the correct minimum ranks. The measured times range from approximately 0.099 to 0.134 milliseconds.

These measurements verify correctness and demonstrate low execution times for small inputs. However, the differences are too small to establish a reliable performance trend from single runs. Larger arrays, more ranks, and repeated measurements would provide a stronger assessment of scalability.

Selected Correctness Results

For $P = 4$ and $N = 4$, the input arrays were

$$A_0 = \{0, 3, 6, 9\},$$
$$A_1 = \{7, 10, 13, 16\},$$
$$A_2 = \{14, 17, 20, 0\},$$
$$A_3 = \{21, 1, 4, 7\}.$$

The resulting minimum values and their originating ranks were

$$\text{Minimum values} = \{0, 1, 4, 0\},$$
$$\text{Minimum ranks} = \{0, 3, 3, 2\}.$$

For the assignment's example with $P = 3$ and $N = 4$,

$$A_0 = \{1, 9, 3, 4\},$$
$$A_1 = \{5, 6, 7, 2\},$$
$$A_2 = \{9, 8, 6, 1\},$$

the implementation returned

$$\text{Minimum values} = \{1, 6, 3, 1\},$$
$$\text{Minimum ranks} = \{0, 1, 0, 2\}.$$

Both tests passed the independent correctness check. “‘